

Securing Passwords Using Docker Compose Secrets

To Begin with the Lab

Summary of the Lab

In this set of labs, we learned how to improve the security and flexibility of Docker Compose deployments. We secured MySQL passwords using Docker Compose Secrets and Docker Swarm Secrets, eliminating the need to hardcode sensitive credentials. We then used environment variables to dynamically select different Docker image versions and configured multiple environment files for Development and Production deployments. These techniques help create secure, reusable, and environment-specific Docker Compose configurations while simplifying application deployment and management.

- Create a new project directory.
- Create the following files:
 - docker-compose.yaml
 - root_password.txt
- Store the MySQL root password inside root_password.txt.
- root_password.txt

1234

- Define the secret in docker-compose.yaml.
- docker-compose.yaml

```
services:
  mysql:
    image: mysql:8.0
    container_name: mysql-instance

    ports:
      - "3306:3306"

    environment:
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/root_password

    secrets:
      - root_password
```

secrets:

root_password:

file: ./root_password.txt

```
ubuntu@ip-172-31-45-84:~$ mkdir pass
ubuntu@ip-172-31-45-84:~$ cd pass
ubuntu@ip-172-31-45-84:~/pass$ nano root_password.txt
ubuntu@ip-172-31-45-84:~/pass$ nano docker-compose.yaml
ubuntu@ip-172-31-45-84:~/pass$
```

- Configure MySQL to read the password from the secret file.
- Start the service.

```
sudo docker compose up -d
```

Verify the running container.

```
sudo docker ps
```

```
ubuntu@ip-172-31-45-84:~/pass$ sudo docker compose up -d
[*] Running 2/2
✓ Network pass default Created 0.1s
✓ Container mysql-instance Started 0.4s
ubuntu@ip-172-31-45-84:~/pass$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
9b77021b5b95   mysql:8.0 "docker-entrypoint.s..." 5 seconds ago Up 4 seconds 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp  mysql-ins
tance
ubuntu@ip-172-31-45-84:~/pass$
```

Inspect the container.

```
sudo docker inspect mysql-instance
```

Verify that the password is referenced as a file instead of appearing as plain text.

Connect to MySQL Workbench and verify the password.

```
ubuntu@ip-172-31-45-84:~/pass$ sudo docker exec -it mysql-instance mysql -uroot -p1234
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Using Docker Swarm Secrets

- Initialize Docker Swarm.

```
sudo docker swarm init
```

- Create the Docker secret.

```
sudo docker secret create root_password root_password.txt
```

```
ubuntu@ip-172-31-45-84:~/pass$ sudo docker swarm init
Swarm initialized: current node (taepsou2rrni2hpxdgbaer07) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-607546ufmmhlv9uwlvtv9zrlpd38gedk5ome3b0lw9dqampzw-3gh27tq9igb05rxl2v13voaxl 172.31.45.84:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
ubuntu@ip-172-31-45-84:~/pass$ sudo docker secret create root_password root_password.txt
nncpanrg3yoy5ks44bq6nlzha
ubuntu@ip-172-31-45-84:~/pass$
```

- Delete the password file.

```
rm root_password.txt
```

- Modify docker-compose.yml to use the external secret.
- Remove container_name from the Compose file.

```
services:
```

```
mysql:
```

```
image: mysql:8.0
```

```
ports:
```

```
- "3306:3306"
```

```
environment:
```

```
MYSQL_ROOT_PASSWORD_FILE: /run/secrets/root_password
```

```
secrets:
```

```
- root_password
```

```
secrets:
```

```
root_password:
```

```
external: true
```

- Deploy the Docker Stack.

```
sudo docker stack deploy -c docker-compose.yaml mysql-stack
```

- Verify the services.

```
sudo docker stack services mysql-stack
```

```
ubuntu@ip-172-31-45-84:~/pass$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-45-84:~/pass$ sudo docker stack deploy -c docker-compose.yaml mysql-stack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network mysql-stack default
Creating service mysql-stack mysql
ubuntu@ip-172-31-45-84:~/pass$ sudo docker stack services mysql-stack
ID            NAME                MODE        REPLICAS  IMAGE      PORTS
jwdqdir5xyp0  mysql-stack_mysql   replicated  1/1        mysql:8.0  *:3306->3306/tcp
ubuntu@ip-172-31-45-84:~/pass$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
8c2fd740d9fa  mysql:8.0  "docker-entrypoint.s..." About a minute ago   Up About a minute   3306/tcp, 33060/tcp  mysql-stack_mysql.1.2qs6m72of6tr2u
qnsavncnug9
ubuntu@ip-172-31-45-84:~/pass$
```

- Connect to MySQL Workbench and verify the password.

```
sudo docker exec -it <container_id> mysql -u root -p
```

- example

```
sudo docker exec -it 8c2fd740d9fa mysql -u root -p
```

- In case you have forgot your password after deleting the file then run

```
sudo docker exec 8c2fd740d9fa cat /run/secrets/root_password
```

```
ubuntu@ip-172-31-45-84:~/pass$ sudo docker exec 8c2fd740d9fa cat /run/secrets/root_password
1234
ubuntu@ip-172-31-45-84:~/pass$ sudo docker exec -it 8c2fd740d9fa mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> exit
Bye
ubuntu@ip-172-31-45-84:~/pass$
```

- Your compose file confirms you're **not using MYSQL_ROOT_PASSWORD directly**. Instead, you're using a Docker Secret:
- environment:
- MYSQL_ROOT_PASSWORD_FILE: /run/secrets/root_password
- That means the root password is the **contents of the Docker secret** named root_password.
- Remove the stack.

```
sudo docker stack rm mysql-stack
```

- Leave Docker Swarm.

```
sudo docker swarm leave --force
```

```
ubuntu@ip-172-31-45-84:~/pass$  
ubuntu@ip-172-31-45-84:~/pass$  
ubuntu@ip-172-31-45-84:~/pass$  
ubuntu@ip-172-31-45-84:~/pass$ sudo docker stack rm mysql-stack  
Removing service mysql-stack_mysql  
Removing network mysql-stack_default  
ubuntu@ip-172-31-45-84:~/pass$ sudo docker swarm leave --force  
Node left the swarm.  
ubuntu@ip-172-31-45-84:~/pass$
```

Using Environment Variables in Docker Compose

- Use the previous php directory that we have created earlier
- Delete all the previous php image that we have created and build the image V1 and V2 again.
- Create a .env file.
- Define the image tag variable.

```
GNU nano 8.7.1
```

```
app=v1
```

```
ubuntu@ip-172-31-45-84:~/php$ nano .env  
ubuntu@ip-172-31-45-84:~/php$ ls  
Dockerfile  docker-compose.yaml  index.php  sample.txt  
ubuntu@ip-172-31-45-84:~/php$ ls -a  
.  ..  .env  Dockerfile  docker-compose.yaml  index.php  sample.txt  
ubuntu@ip-172-31-45-84:~/php$
```

- Modify docker-compose.yaml to reference the variable.

```
version: "3.8"
```

```
services:
```

```
  webapp:
```

```
    build: .
```

```
    image: phpapp:${app}
```

```
    container_name: php-webapp
```

```
ports:
```

```
- "8080:80"
```

- Start the container.

```
sudo docker compose up -d
```

- Verify Version 1.

Hello from Cloudfreeks!

This is Version 1

Current Time: 2026-07-15 05:56:14

PHP Version: 8.2.32

Contents of sample.txt

Hello, Cloudfreeks!

- Stop the container.

```
sudo docker compose down
```

- Modify the application.

```
<?php
echo "<h1>Hello from Cloudfreeks!</h1>";

echo "<h1>This is Version 2</h1>";

echo "<p><strong>Current Time:</strong> " . date("Y-m-d H:i:s") . "</p>";

echo "<p><strong>PHP Version:</strong> " . phpversion() . "</p>";
```

- Build Version 2 of the PHP image.
- Change the variable from v1 to v2.

```
GNU nano 8.7.1
app=v2
```

- Start the container again.

```
sudo docker compose up -d
```

- Verify Version 2.

```
[+] Running 3/3
✓ webapp Built
✓ Network php_default Created
✓ Container php-webapp Started
ubuntu@ip-172-31-45-84:~/php$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
php:8.2-apache	2a195673289c	714MB	183MB	
phpapp:v1	cb5a1ceea762	707MB	176MB	
phpapp:v2	9b7cdcd28ca6	707MB	176MB	

```
ubuntu@ip-172-31-45-84:~/php$
```

Hello from Cloudfreeks!

This is Version 2

Current Time: 2026-07-15 05:54:28

PHP Version: 8.2.32

Contents of sample.txt

Hello, Cloudfreeks!

- Code for Dockerfile

```
FROM php:8.2-apache
WORKDIR /var/www/html
COPY . .
EXPOSE 80
```

- Code for index.php

```
<?php
echo "<h1>Hello from Cloudfreeks!</h1>";

echo "<h1>This is Version 1</h1>";

echo "<p><strong>Current Time:</strong> " . date("Y-m-d H:i:s") . "</p>";

echo "<p><strong>PHP Version:</strong> " . phpversion() . "</p>";

echo "<hr>";
```

```
echo "<h3>Contents of sample.txt</h3>";

if (file_exists("sample.txt")) {
    echo "<pre>";
    echo htmlspecialchars(file_get_contents("sample.txt"));
    echo "</pre>";
} else {
    echo "<p style='color:red;'>sample.txt not found!</p>";
}

?>
```

Managing Multiple Environments Using Environment Files

- Create a config directory.
- Create:

```
nano .env.dev
```

- Configure the Development environment variables.

```
app=dev
```

```
nano .env.prod
```

```
app=prod
```

- Configure the Production environment variables.
- Start the Development environment.

```
sudo docker compose --env-file config/.env.dev up -d
```

Hello from Cloudfreeks!

This is Development Version

Current Time: 2026-07-15 06:21:09

PHP Version: 8.2.32

Contents of sample.txt

- Stop the containers.

`sudo docker compose down`

- Start the Production environment.

`docker compose --env-file config/.env.prod up -d`

Verify the Production application.

Hello from Cloudfreeks!

This is Production Version

Current Time: 2026-07-15 06:25:01

PHP Version: 8.2.32

Contents of sample.txt

Hello, Cloudfreeks!